

Spring Boot & RESTful Web Services

Comprehensive Technical Implementation Guide

An in-depth code documentation and conceptual reference for building Enterprise Java Microservices. Covers Dependency Injection, Web Endpoints, Service Architectures, Exception Handling, and JWT Security Filters.

Target Architecture: Spring Boot, Spring Web, Spring Security, JWT

Build Tool: Apache Maven

Date: July 2026

1. Create a Spring Web Project using Maven

Spring Boot streamlines Java enterprise development by providing automatic configuration, starter dependencies, and an embedded servlet container. This eliminates boilerplate XML setup and minimizes project initialization overhead.

Project Initialization and Structure

Using the **Spring Initializr** (<https://start.spring.io>), projects can be pre-configured with structural scaffolding. The foundation layout of a standard Maven-based Spring Boot application is organized as follows:

- `src/main/java` : Holds the corporate domain package structure and executable application source files.
- `src/main/resources` : Stores static assets, public templates, and external properties files (e.g., `application.properties`).
- `src/test/java` : Reserved for test definitions such as JUnit suites and integration test harnesses.
- `pom.xml` : The Maven Project Object Model file declaring transitive libraries, project metadata, and build configurations.

Project Blueprint Definition

The initialization parameters used for generating this baseline environment are:

Parameter	Configuration Value
Group ID	<code>com.cognizant</code>
Artifact ID	<code>spring-learn</code>
Dependencies	Spring Web, Spring Boot DevTools

Bootstrap Configuration & Compilation

To execute dependency retrieval and project building across authenticated corporate networks, standard proxy flags must be provided during compilation. Run the following command from the terminal:

```
mvn clean package -Dhttp.proxyHost=proxy.cognizant.com -Dhttp.proxyPort=6050 -  
Dhttps.proxyHost=proxy.cognizant.com -Dhttps.proxyPort=6050 -Dhttp.proxyUser=123456
```

The Bootstrap Class Implementation

The main bootstrap code resides inside `SpringLearnApplication.java`. The `@SpringBootApplication` annotation triggers three key automatic capabilities: `@SpringBootConfiguration`, `@EnableAutoConfiguration`, and `@ComponentScan`.

```
package com.cognizant.springlearn;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;

@SpringBootApplication
public class SpringLearnApplication {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(SpringLearnApplication.class);

    public static void main(String[] args) {
        LOGGER.info("START: Launching SpringLearnApplication Bootstrap Framework");

        // Execute the embedded environment and capture context references
        ApplicationContext context = SpringApplication.run(SpringLearnApplication.class,
args);

        LOGGER.info("END: Bootstrap framework initialized completely");
    }
}
```

2. Spring Core – Load Country from Spring Configuration XML

While modern Spring applications leverage annotation-driven wiring, legacy integration and certain external components require XML-based configuration metadata inside the Inversion of Control (IoC) Container.

The Domain Model Entity

The `Country` class below models localized geographic regions for the travel reservation subsystem. It includes explicit internal logging within lifecycle methods to track container interactions.

```

package com.cognizant.springlearn;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class Country {
    private static final Logger LOGGER = LoggerFactory.getLogger(Country.class);

    private String code;
    private String name;

    // Explicit Default No-Arg Constructor used by Setter Injection
    public Country() {
        LOGGER.debug("Inside Country Constructor: Component instantiated via
reflection.");
    }

    public String getCode() {
        LOGGER.debug("Accessing field [code]: {}", this.code);
        return code;
    }

    public void setCode(String code) {
        LOGGER.debug("Mutating field [code] -> Setter Injection value: {}", code);
        this.code = code;
    }

    public String getName() {
        LOGGER.debug("Accessing field [name]: {}", this.name);
        return name;
    }

    public void setName(String name) {
        LOGGER.debug("Mutating field [name] -> Setter Injection value: {}", name);
        this.name = name;
    }

    @Override
    public String toString() {
        return "Country [code=" + code + ", name=" + name + "]";
    }
}

```

The XML Configuration Registry (country.xml)

This registry defines explicit mapping rules for individual beans. Property mutations are processed by the container via regular Java setter methods.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    https://www.springframework.org/schema/beans/spring-beans.xsd">

  <!-- Singleton Scope configuration by default -->
  <bean id="country" class="com.cognizant.springlearn.Country">
    <property name="code" value="IN" />
    <property name="name" value="India" />
  </bean>

</beans>
```

Extracting Beans from the ClassPath Application Context

The context scans the classpath for the XML specification, builds an internal definition graph, processes instantiation, hooks up setters, and registers references.

```
public static void displayCountry() {
    LOGGER.info("START: displayCountry routine initiated");

    // Instantiate programmatic resource container parsing XML mapping metadata
    ApplicationContext context = new ClassPathXmlApplicationContext("country.xml");

    // Retrieve reference safely with strict casting parameters
    Country country = context.getBean("country", Country.class);

    LOGGER.debug("Country Object Details: {}", country.toString());
    LOGGER.info("END: displayCountry execution finalized");
}
```

3. Hello World RESTful Web Service

RESTful components decouple user-facing client frameworks from backend logic through stateless HTTP exchanges.

The REST Controller Implementation

The `@RestController` stereotype annotation identifies the component as an active request handler, implicitly formatting all returned data payloads directly into the HTTP response body.

```

package com.cognizant.springlearn.controller;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {

    private static final Logger LOGGER = LoggerFactory.getLogger(HelloController.class);

    /**
     * Handles inbound GET queries mapped to the base web context.
     * @return Raw text response string.
     */
    @GetMapping("/hello")
    public String sayHello() {
        LOGGER.info("START: Inbound REST execution request intercepted at endpoint [/hello]");

        String response = "Hello World!!";

        LOGGER.info("END: Terminating handler routine, resolving outbound raw text payload");
        return response;
    }
}

```

Port Conflict Resolution Tip: If port `8080` is already bound by another application container, append `server.port=8083` inside your `src/main/resources/application.properties` configuration directory to shift target operations.

4. REST - Country Web Service

Enterprise architectures combine disparate configurations by pulling objects from legacy XML definitions and delivering them downstream as JSON arrays or map schemas.

The XML-to-JSON Controller Handler Bridge

The `CountryController` uses standard `ClassPathXmlApplicationContext` processing parameters to resolve targets on demand. Spring's underlying Jackson binding engine automatically transforms the POJO into structured JSON strings.

```

package com.cognizant.springlearn.controller;

import com.cognizant.springlearn.Country;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class CountryController {

    private static final Logger LOGGER = LoggerFactory.getLogger(CountryController.class);

    /**
     * Resolves localized context references from XML configurations.
     * @return Country payload serialized automatically by Jackson messaging filters.
     */
    @RequestMapping(value = "/country", method = RequestMethod.GET)
    public Country getCountryIndia() {
        LOGGER.info("START: Processing inbound payload extraction request at [/country]");

        ApplicationContext context = new ClassPathXmlApplicationContext("country.xml");
        Country country = context.getBean("country", Country.class);

        LOGGER.info("END: Country entity retrieved successfully; starting JSON marshalling pipeline");
        return country;
    }
}

```

Expected Application Serialization Response

Inbound client queries routed to <http://localhost:8083/country> receive an automated response profile structured as follows:

```

{
  "code": "IN",
  "name": "India"
}

```

5. REST - Get Country Based on Country Code

To pull deep parameters from URI query elements, paths must be designed with dynamic parameters that feed straight into case-insensitive business lookups.

Business Layer Validation Exception Definition

When an unmapped or missing code parameter is provided, the application halts further processing and returns a descriptive HTTP status code.

```
package com.cognizant.springlearn.service.exception;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ResponseStatus;

@ResponseStatus(value = HttpStatus.NOT_FOUND, reason = "Country not found")
public class CountryNotFoundException extends Exception {
    // Custom unchecked exception layer controlling status routing pipelines
    public CountryNotFoundException(String message) {
        super(message);
    }
}
```

The Core Service Definition Subsystem

The `CountryService` class instantiates an aggregate dataset array from XML configurations and filters for code tokens using the Stream API.

```
package com.cognizant.springlearn.service;

import com.cognizant.springlearn.Country;
import com.cognizant.springlearn.service.exception.CountryNotFoundException;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.List;

@Service
public class CountryService {

    @SuppressWarnings("unchecked")
    public Country getCountry(String code) throws CountryNotFoundException {
        ApplicationContext context = new ClassPathXmlApplicationContext("country.xml");

        // Extract multi-bean reference collection array
        List<Country> countries = context.getBean("countryList", ArrayList.class);

        return countries.stream()
            .filter(c -> c.getCode().equalsIgnoreCase(code))
            .findFirst()
            .orElseThrow(() -> new CountryNotFoundException("Target code " + code + "
missing from cluster"));
    }
}
```


Dynamic Parameter Controller Implementation

The `@PathVariable` annotation maps placeholders within the path sequence directly to typed arguments in the service layer method signature.

```
// Added inside com.cognizant.springlearn.controller.CountryController

@Autowired
private CountryService countryService;

@GetMapping("/countries/{code}")
public Country getCountry(@PathVariable String code) throws CountryNotFoundException {
    LOGGER.info("START: Resolving dynamic path selection variable for token code [{ }]",
        code);

    Country country = countryService.getCountry(code);

    LOGGER.info("END: Target record resolved successfully matching value key [{ }]", code);
    return country;
}
```

6. Create Authentication Service That Returns JWT

Basic Authentication requires credentials to accompany every network request, which creates a large attack surface. JSON Web Tokens (JWT) mitigate this by exchanging valid long-lived credentials for a short-lived, digitally signed cryptographic string.

Securing Endpoints via Declarative Rules

The security policy below mandates Basic Authentication for the initial token exchange, but routes all downstream data queries through a dedicated JWT validation filter.

```

package com.cognizant.springlearn.security;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.authentication.builders.AuthenticationManagerBuilder;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configuration.WebSecurityConfigurerAdapter;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;

@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {

    private static final Logger LOGGER = LoggerFactory.getLogger(SecurityConfig.class);

    @Override
    protected void configure(AuthenticationManagerBuilder auth) throws Exception {
        auth.inMemoryAuthentication()
            .withUser("admin").password(passwordEncoder().encode("pwd")).roles("ADMIN")
            .and()
            .withUser("user").password(passwordEncoder().encode("pwd")).roles("USER");
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Override
    protected void configure(HttpSecurity httpSecurity) throws Exception {
        httpSecurity.csrf().disable()
            .httpBasic().and()
            .authorizeRequests()
            .antMatchers("/authenticate").hasAnyRole("USER", "ADMIN")
            .anyRequest().authenticated().and()
            .addFilter(new JwtAuthorizationFilter(authenticationManager()));
    }
}

```

The Token Generation & Interception API Controller

The token generation endpoint parses credentials out of incoming HTTP Authorization header tokens, decodes them via Base64, verifies the user, and signs a programmatic token structure using HS256.

```

package com.cognizant.springlearn.controller;

import io.jsonwebtoken.JwtBuilder;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestHeader;
import org.springframework.web.bind.annotation.RestController;

import java.util.Base64;
import java.util.Date;
import java.util.HashMap;
import java.util.Map;

@RestController
public class AuthenticationController {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(AuthenticationController.class);

    @GetMapping("/authenticate")
    public Map<String, String> authenticate(@RequestHeader("Authorization") String
authHeader) {
        LOGGER.info("START: Processing token extraction execution protocol");
        LOGGER.debug("Target Authorization Parameter Payload: {}", authHeader);

        String user = getUser(authHeader);
        String token = generateJwt(user);

        Map<String, String> response = new HashMap<>();
        response.put("token", token);

        LOGGER.info("END: JWT created completely; returning authorization payload map");
        return response;
    }

    private String getUser(String authHeader) {
        String encodedCredentials = authHeader.replace("Basic ", "");
        byte[] decodedBytes = Base64.getDecoder().decode(encodedCredentials);
        String credentials = new String(decodedBytes);
        return credentials.split(":")[0]; // Extract user parameter token safely
    }

    private String generateJwt(String user) {
        JwtBuilder builder = Jwts.builder()
            .setSubject(user)
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() + 1200000)) // 20-
minute expiry
            .signWith(SignatureAlgorithm.HS256, "secretkey");
        return builder.compact();
    }
}

```

Verifying Token Operations via cURL

To execute credential validation and fetch a signed bearer token, dispatch the following request through your terminal instance:

```
curl -s -u user:pwd http://localhost:8090/authenticate
```

The service will process the request and return a signed JSON payload structured as follows:

```
{"token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyIiwiaWF0IjoxNTcwMzc5NDc0LCJleHAiOiE1NzAzODAxMzR9.t3LRv1CV-hwKfoqZYlaVQqEUiBlowcWn0ft3tgv0dL0"}
```